



DAY 9 · FULL STACK BOOTCAMP

Connecting PHP with MySQL Saving Data Permanently

Today we stop losing data on refresh. We connect our PHP forms directly to MySQL, build a real Student Registration Database, and display live records inside a Bootstrap table — all in 2.5 hours.

Browser

Registration Form

MySQL

Store Permanently

PHP

Process & Validate

Student Records

Display & Retrieve

OPENING HOOK

What Happens After You Hit Refresh?

Yesterday your form submitted data and displayed it on screen. You hit refresh — and everything disappeared. How do Instagram, Amazon, and WhatsApp remember your data *forever*?

The Problem

PHP variables live only during one request. Once the page reloads, all submitted data is gone. There is no memory between requests.

The Solution

A **database** stores data on disk permanently. MySQL is the industry-standard relational database that PHP talks to natively — and we learn it today.

- ❑ Take 30 seconds to discuss with your neighbor: where do you think your Instagram profile data actually lives?



TODAY'S MISSION

Student Registration Database

By the end of this session you will have a fully working web application — not a demo, not a mockup — a real app that saves and retrieves student records from MySQL.



Registration Form

HTML + Bootstrap form collecting student details



PHP Backend

Validates input and runs SQL queries



MySQL Storage

Permanently saves every student record



Bootstrap Table

Displays all records live in the browser



⚡ RAPID FIRE RECAP

Pair Up — 3 Minutes

Turn to your neighbor. Take turns answering each question out loud. No peeking at notes. This is your warm-up before we write real code.

What is POST?

How is it different from GET?

What is include()?

Why do we use it instead of copy-pasting code?

Why validate?

What bad things happen without server-side validation?

PHP vs JavaScript?

Where does each language execute?

What is localhost?

What software is running behind it?

What Is a Database? — Concept + Live Coding

Think of a MySQL database as a collection of **spreadsheets that never close**. Each spreadsheet is a **table**. Each column is a **field**. Each row is a **record**. The **Primary Key** (id) uniquely identifies every row — just like a roll number identifies every student.

Database

student_management — the container for all tables

Table

students — stores one row per registered student

Primary Key

id — AUTO_INCREMENT, unique, never NULL

Live Coding — do this now:

1. Open `localhost/phpmyadmin` in your browser
2. Click **New** in the left sidebar
3. Enter database name: `student_management`
4. Collation: `utf8_general_ci`
5. Click **Create**

📋 You should see the new database appear in the left panel. Screenshot it — we'll refer to it later.

Create Your Students Table

Inside `student_management`, create a table named `students` with the fields below. Use phpMyAdmin's table creation UI — click each field, set the type, and mark `id` as PRIMARY KEY with AUTO_INCREMENT.

id

INT · AUTO_INCREMENT · PRIMARY KEY

name

VARCHAR(100) · NOT NULL

email

VARCHAR(150) · UNIQUE

branch

VARCHAR(50)

cgpa

DECIMAL(3,2)

Challenge

Add two extra fields: `phone` (VARCHAR 15) and `city` (VARCHAR 60). Choose appropriate data types and explain your choice to your neighbor.

Bonus

Add a `created_at` field of type `TIMESTAMP` with a default value of `CURRENT_TIMESTAMP`. This auto-records when each student was registered.

Connecting PHP to MySQL — `db_connect.php`

Every PHP page that needs database access will include one file: `db_connect.php`. We use `mysqli_connect()` — the MySQLi extension built into PHP. We test the connection immediately so we catch errors early.

Live Coding — Create `db_connect.php`

```
<?php
$host      = "localhost";
$user      = "root";
$password  = "";
$database  = "student_management";

$conn = mysqli_connect($host, $user, $password,
    $database);

if (!$conn) {
    die("Connection Failed: " . mysqli_connect_error());
}
// echo "Connection Successful!";
?>
```

Understanding the Parameters

- **localhost**
MySQL server address — your own machine via XAMPP
- **root / ""**
Default XAMPP username and blank password
- **student_management**
The database we just created in phpMyAdmin
- **die()**
Stops the script and shows the error if connection fails

Connect Your Project to MySQL

Create `db_connect.php` in your project folder, paste the connection code, and include it at the top of your `index.php` using `include('db_connect.php');`. Load the page in your browser and confirm no errors appear.

01

Create the file

New file in VS Code: `db_connect.php` inside your project root

02

Paste & save

Add the `mysqli_connect()` code with your correct database name

03

Include it

Add `<?php include('db_connect.php'); ?>` at the very top of `index.php`

04

Test in browser

Open `localhost/yourproject/index.php` — no errors = success

Challenge

Uncomment the success echo and wrap it in a Bootstrap alert-success div so a green banner appears when the connection works.

Bonus

Use `mysqli_get_host_info($conn)` to display the current database server info inside the alert banner.



MODULE 3

Inserting Data — Concept + Live Coding

Take the registration form you built on Day 8. When it's submitted via POST, we grab each field, build a SQL INSERT query, and execute it using `mysqli_query()`. The data is now **permanently** saved in MySQL.

Live Coding — `process_form.php`

```
<?php
include('db_connect.php');

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $name    = mysqli_real_escape_string($conn, $_POST['name']);
    $email   = mysqli_real_escape_string($conn, $_POST['email']);
    $branch  = mysqli_real_escape_string($conn, $_POST['branch']);
    $cgpa    = $_POST['cgpa'];

    $sql = "INSERT INTO students (name, email, branch, cgpa)
           VALUES ('$name', '$email', '$branch', '$cgpa')";

    if (mysqli_query($conn, $sql)) {
        echo "Student Registered Successfully!";
    } else {
        echo "Error: " . mysqli_error($conn);
    }
}
?>
```

mysqli_real_escape_string()

Sanitizes user input to prevent SQL injection — always use it on string fields

INSERT INTO

SQL command to add a new row. Columns listed in () must match values in VALUES ()

mysqli_query()

Executes the SQL on the connected database. Returns true/false

After inserting

Open phpMyAdmin → Browse students table. You should see your row appear!

 MISSION 3 CHALLENGE

Save Student Information to MySQL

Wire your Day 8 registration form to the database. After submission, the student's details should appear in the `students` table when you click Browse in phpMyAdmin. Verify it works before moving on.

✓ Required Fields

- name (VARCHAR)
- email (VARCHAR)
- branch (VARCHAR)
- cgpa (DECIMAL)

Challenge

Prevent duplicate email registrations. Check if the email already exists before inserting. Show a friendly Bootstrap warning if the email is taken.

Bonus

After successful insertion, run a `SELECT COUNT(*)` query and display: *"You are student #N in our system!"* using a Bootstrap info alert.

 After each insert, open phpMyAdmin and click Browse on the students table to visually confirm your data saved correctly.

Displaying Records — Concept + Live Coding

Now the exciting part — we retrieve every student from MySQL and render them inside a professional Bootstrap table. We use `mysqli_fetch_assoc()` inside a while loop to process each row one at a time.

Live Coding — students.php

```
<?php include('db_connect.php'); ?>
<table class="table table-bordered table-hover">
  <thead class="table-dark">
    <tr>
      <th>ID</th><th>Name</th>
      <th>Branch</th><th>CGPA</th>
    </tr>
  </thead>
  <tbody>
    <?php
    $sql = "SELECT * FROM students";
    $result = mysqli_query($conn, $sql);
    while ($row = mysqli_fetch_assoc($result)) {
      echo "<tr>";
      echo "<td>" . $row['id'] . "</td>";
      echo "<td>" . $row['name'] . "</td>";
      echo "<td>" . $row['branch'] . "</td>";
      echo "<td>" . $row['cgpa'] . "</td>";
      echo "</tr>";
    }
    ?>
  </tbody>
</table>
```

→ **SELECT ***

Fetches every column from every row in the students table

→ **mysqli_query()**

Executes the SELECT and returns a result object

→ **mysqli_fetch_assoc()**

Returns one row as an associative array.
Returns false when no more rows remain — ending the while loop

→ **\$row['column']**

Access each field by its column name exactly as defined in MySQL

Build the Student List Page

Create `students.php` — a dedicated page that fetches and displays every registered student in a clean Bootstrap table. This is the second half of your full-stack registration system.

Display Columns

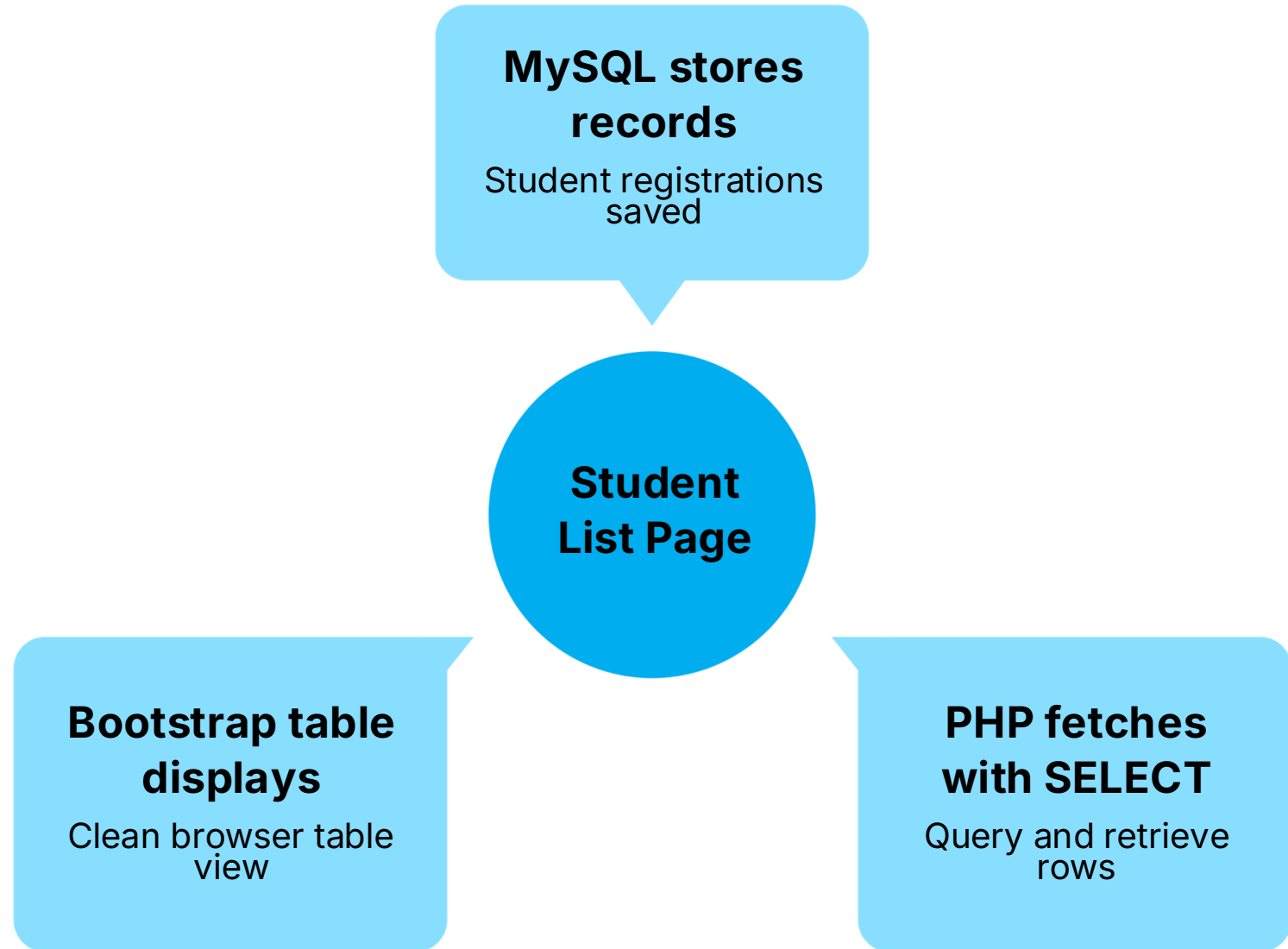
ID · Name · Branch · CGPA — styled with Bootstrap `table-hover` and `table-bordered`

🏆 Challenge

Use a PHP `if` statement inside the while loop to add class `table-success` to any row where `CGPA > 8.0`

★ Bonus

Below the table, display: *"Total Students: N"* using `mysqli_num_rows($result)`



Spot the Bug

The application below is broken. In pairs, examine each issue and identify the root cause before the trainer reveals the answer. Write your diagnosis in your notebook.

Wrong SQL Syntax

`INSRT INTO students` — typo in the SQL keyword causes a fatal query error with no helpful message

Wrong Column Name

`$row['Name']` vs `$row['name']` — MySQL column names are case-sensitive in PHP arrays

Missing Semicolon

PHP statement without `;` causes a parse error that breaks the entire page rendering

Missing Fetch Loop

Calling `mysqli_query()` without `mysqli_fetch_assoc()` — the query runs but no data is ever read

Broken Bootstrap Table

Opening `<table>` tag inside the PHP while loop — creates a new table for every row instead of one unified table

30-Minute Hackathon — Student Registration Portal

Timer: 30 Minutes

Core Requirements ✓

- Registration Form (Bootstrap)
- Server-side validation (PHP)
- Save records to MySQL
- Display all records in Bootstrap table
- No crashes, no SQL errors

Bonus Features ★

- Bootstrap success alert on registration
- Record counter ("Total: N students")
- Highlight top CGPA rows in green
- Duplicate email prevention
- Professional navbar with project name

 This is a real deliverable. Every feature you add here goes directly onto your GitHub portfolio. Build it like a client is watching.



 DEMO DAY

Student Showcase — 2 Minutes Per Team

Each team presents their completed Student Registration Portal to the class. Be ready to demo live — the trainer will ask one follow-up question on SQL or PHP to every team.



Show Your Database

Open phpMyAdmin and show the students table with real records. Explain your schema choices — data types, primary key, UNIQUE constraints.



Demo the Form

Fill in the registration form live. Switch to phpMyAdmin and refresh — show the new row appearing in real time. This is the magic moment.



Show Student List

Navigate to your students.php page. Show all records in the Bootstrap table. Highlight any bonus features — row coloring, record count, alerts.



Biggest Challenge

Share the toughest bug you hit and how you solved it. Every team has one — this is the most valuable learning for the whole class.

Interview-Ready Questions

These questions come up in every PHP/MySQL junior developer interview. Answer each one out loud without looking at your code. If you can explain it in plain English, you truly understand it.

1

Why use a database?

Variables disappear after each request. A database persists data on disk so records survive across sessions, users, and server restarts.

2

Table vs. Database?

A database is the container. A table is one organized dataset inside it — like an Excel workbook vs. a single sheet.

3

Purpose of Primary Key?

Guarantees every row has a unique identity. Enables fast lookups, relationships between tables, and prevents duplicate records.

4

Purpose of `mysqli_connect()`?

Opens a live connection between your PHP script and the MySQL server. Without it, no query can be sent or received.

5

What does `SELECT *` return?

All columns and all rows from the specified table. In production, always specify column names instead of `*` for performance.



📅 ASSIGNMENT

Improve Your Project Tonight

Tonight's assignment builds on what you finished in class. Push your enhanced project to GitHub before Day 10 starts. The trainer will check commits at the beginning of tomorrow's session.

Required Additions

- Add `photo` column (VARCHAR for filename)
- Add `address` (TEXT)
- Add `course` (VARCHAR 100)
- Add `date_registered` (TIMESTAMP)
- Update Bootstrap table to show new fields

Push to GitHub

1. `git add .`
2. `git commit -m "Day 9: PHP-MySQL integration"`
3. `git push origin main`

Your GitHub profile is your resume. Every push you make tonight is visible to every recruiter who looks you up.

What You Built Today

In 2.5 hours you went from knowing PHP basics to building a complete data-driven web application. That is a massive leap — and it is now permanently on your GitHub.



✓ MySQL Database

Created `student_management` with a properly typed `students` table



✓ PHP Connection

Built `db_connect.php` using `mysqli_connect()` with error handling



✓ INSERT Records

Processed form data and saved students to MySQL with SQL injection protection



✓ Display Records

Fetches all rows with `SELECT` and rendered them in a live Bootstrap table

Today you learned how to **save data**. Tomorrow you'll learn how to **manage it like a real application** — edit, delete, and search records. See you at Day 10. 🚀